

LABORATORIO DI SISTEMI OPERATIVI

System Calls per i processi

fabio.rossi@unipg.it

075 5855005

PID e PPID

```
#include <unistd.h>
pid_t getpid(void) ;
pid_t getppid(void) ;
```

- *getpid()* torna il *process ID*
- *getppid()* torna il *parent process ID*

Ritorno: queste funzioni “*are always successful*”
(*man*)

- c'è anche *getpgrp()* che torna il *process group ID*

Creazione processi: *fork()*

```
#include <unistd.h>
pid_t fork(void);
```

- crea un nuovo processo ***duplicando*** il chiamante
 - tuttavia alcuni attributi non sono duplicati:
 - *PID, memory locks, segnali pendenti, timer, etc.*
- Il nuovo processo è detto ***child***, il processo chiamante ***parent***
- lo spazio di indirizzamento del nuovo processo è un duplicato di quello del *parent* (è **separato**, ognuno ha il proprio)
- Il *child* eredita una copia della *tabella dei descrittori dei file* aperti del processo padre, ma:
 - le due copie dello stesso *file descriptor* si riferiscono alla stessa voce nella *tabella dei file aperti* (di sistema), cioè condividono:
 - i flag di stato del file
 - il puntatore alla locazione corrente di ogni file
 - Etc. ([vedi man open\(2\)](#) e [man fork\(2\)](#))

Ritorna:

- nel *parent*: il ***PID*** del *child*
- nel *child*: **0**
- **-1** in caso di errore (il *child* non viene creato). ***errno*** viene settata

Creazione processi: *fork()*

- **Esempio visto nella parte teorica (*Prof. Carpi*):**

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;
    /* fork a child process */
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete\n");
    }
    return 0;
}
```

System Calls per i processi

Nome	Cosa fa
fork	crea un nuovo processo
wait	attende finché uno dei <i>child</i> termina
waitpid	attende finché un determinato <i>child</i> cambia stato
waitid	(da <i>Linux 2.6.9</i>) maggiore controllo su quali child attendere
_exit(2), exit(3)	terminano il chiamante.
execve(2)	esegue un programma che rimpiazza il chiamante
exec*(3)	wrappers per execve(2) (sono 6)

wait() e *waitpid()*

```
#include <sys/wait.h>
pid_t wait(int *wstatus);
pid_t waitpid(pid_t pid, int *wstatus, int options)
```

- *wait()* sospende il chiamante finché un *child* termina
- *waitpid()* sospende il chiamante finché un *child* cambia stato (terminato; stoppato da un *signal*; ha ripreso dopo un *signal*); dipende da **options**
- permettono di rilasciare le risorse assegnate al *child*:
 - se un *child* termina e una *wait* non viene eseguita il *child* rimane nello stato di *zombie* (vedi la sezione "*NOTES*" di *man wait(2)*)

Ritorno:

- Il PID del processo il cui stato è cambiato (*waitpid()*) può tornare anche 0 in certe circostanze, vedi *man wait(2)*
- -1 in caso di errore

wait() e *waitpid()*

```
#include <sys/wait.h>
pid_t wait(int *wstatus);
pid_t waitpid(pid_t pid, int *wstatus, int options
```

- se un/il *child* è terminato/ha già cambiato stato tornano immediatamente
- `waitpid(-1, &wstatus, 0)` è equivalente a `wait(&wstatus)`

waitpid()

```
#include <sys/wait.h>
```

```
pid_t waitpid(pid_t pid, int *wstatus, int options
```

- tra i valori che possiamo dare a **pid** ci sono i seguenti:
 - -1: attende per un cambiamento di stato di uno qualsiasi dei child
 - > 0: attende per un cambiamento di stato del *child* il cui *process ID* è uguale a **pid**
 - etc. (vedi *man 2 wait*)

wait() e *waitpid()*

```
#include <sys/wait.h>
pid_t wait(int *wstatus);
pid_t waitpid(pid_t pid, int *wstatus, int options
```

- se **&wstatus** non è *NULL* riceve informazioni sul *child*. Queste informazioni possono essere evinte con delle macro:
 - *WIFEXITED(wstatus)* torna *true* se il *child* è terminato normalmente (*exit()* o *return*)
 - *WEXITSTATUS(wstatus)* ritorna il codice di uscita del *child*
 - Etc. (*man 2 wait*)

Creazione processi: *execve(2)*

```
#include <unistd.h>
```

```
int execve(const char *pathname, char *const argv[],  
           char *const envp[]);
```

- *fork()* ed *execve()* (o le altre funzioni *exec(3)*) sono spesso utilizzate insieme
- *execve()* **rimpiazza il programma eseguito dal processo chiamante col programma specificato da *pathname***
- *Stack*, *heap* e *data segments* sono reinizializzati

Ritorno:

- **se non c'è errore non ritorna !**
- -1 in caso di errore. ***errno*** settata

Creazione processi: *execve(2)*

```
#include <unistd.h>
```

```
int execve(const char *pathname, char *const argv[],  
           char *const envp[]);
```

- ***pathname***: path del programma da eseguire. Può essere un eseguibile o uno *script* (che contenga la *sha-bang*, vedi slides sullo *scripting* in *Bash*...)
- ***argv***: array di puntatori a stringa che contiene gli argomenti che verranno passati al nuovo programma
- ***envp***: array di puntatori a stringhe, nella forma *nome=valore*, che verranno passate come *environment* al nuovo programma

Creazione processi: funzioni *exec*(3)

```
#include <unistd.h>
extern char **environ;

int execl(const char *pathname, const char *arg, ...
          /* (char *) NULL */);
int execlp(const char *file, const char *arg, ...
          /* (char *) NULL */);
int execl_e(const char *pathname, const char *arg, ...
            /*, (char *) NULL, char *const envp[] */);
int execv(const char *pathname, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[],
            char *const envp[]); #include <unistd.h>
```

(da man 3 exec)

Creazione processi: funzioni *exec(3)*

- rimpiazzano il processo chiamante con un nuovo processo
- Chiamano la sottostante *execve(2)*
- Il primo argomento è il nome del file da eseguire

Ritorno:

- se non c'è errore non ritornano
- -1 in caso di errore. ***errno*** settata

exec(3)*

- raggruppabili in base alle lettere che seguono il prefisso “*exec*” (*man 3 exec*):
 - **l**: (**execl()**, **execvp()**, **execle()**) gli argomenti da passare al programma chiamato sono specificati come lista
 - **v**: (**execv()**, **execvp()**, **execvpe()**) gli argomenti da passare al programma chiamato sono specificati in un array (*vector*)
 - **e**: (**execle()**, **execvpe()**) *envp* specifica l'*environment*. Gli altri *wrappers* prendono l'*environment* dalla variabile esterna *environ*
 - **p**: (**execvp()**, **execvp()**, **execvpe()**) se il primo argomento del wrapper non è un path assoluto il programma da eseguire è cercato in base alla variabile di ambiente *PATH*

execl(3)

```
#include <unistd.h>
int execl(const char *pathname, const char *arg, ...
          /* (char *) NULL */);
```

- *pathname* è il programma da eseguire
- *const char *arg* ed i susseguenti argomenti (in numero variabile) equivalgono ad *arg0*, *arg1*, ..., *argn* e rappresentano gli argomenti del programma da eseguire
- il primo argomento per convenzione è il nome del programma
- l'ultimo argomento dovrebbe essere NULL e castato:
*(char *) NULL*

Ritorno:

- se non c'è errore non ritorna
- -1 in caso di errore. **errno** settata

_exit(2)

```
#include <unistd.h>
void _exit(int status);
```

- termina il chiamante "immediately"
- i *file descriptors* sono chiusi
- il *parent ID* di eventuali *child* diventa *init* (o il più vicino "*subreaper process*"... se interessati vedere *man prctl(2)*)

Ritorno:

- Non c'è (ovviamente).....
- ... ma il processo *parent* può ottenere il byte meno significativo di *status* (**status & 0xFF**) con *wait(2)*

`_exit(2)` vs `exit(3)`

```
#include <stdlib.h>
void exit(int status);
```

- termina il chiamante
- `exit(3)` fa tutto quello che fa `_exit(2)`, ma inoltre:
 - chiama eventuali funzioni registrate con `atexit(3)` e `on_exit(3)`
 - esegue il *flush* dei buffer di I/O relativi alle funzioni di libreria C per l'I/O (`stdio(3) streams`) e chiude i relativi *stream*
- Lo standard C definisce `EXIT_SUCCESS` e `EXIT_FAILURE` (per portabilità) come valori di uscita

Ritorno:

- Non c'è (ovviamente).....
- ... ma il processo *parent* può ottenere il byte meno significativo di *status* (`status & 0xFF`) con `wait(2)`